Laboratorio 8: Lista de Estudiantes

Contexto y objetivos

En este laboratorio pondrá en práctica conceptos fundamentales de la programación funcional en Kotlin, utilizando un proyecto de Android Studio (versión LadyBug) como entorno de trabajo. Aunque el proyecto será una app Android mínima, todo el código se ejecutará en consola (Logcat), sin interfaz gráfica. Explorará lambdas, funciones de orden superior, funciones de extensión y propiedades de extensión.

Requisitos previos

- Conocimiento básico de Kotlin (sintaxis, clases, funciones).
- Android Studio instalado (versión Arctic Fox o superior, idealmente LadyBug).
- Saber crear un proyecto básico con una Activity vacía.
- Conocer cómo abrir y usar Logcat.

Descripción general

Creará un programa que gestiona una lista de estudiantes con sus calificaciones. Aplicará funciones de orden superior para filtrar y transformar datos, definirá funciones de extensión para enriquecer tipos existentes, y utilizará propiedades de extensión para obtener información derivada. Todo el procesamiento se mostrará en Logcat.

Paso a paso detallado

1. Abrir Android Studio y crea un nuevo proyecto con las siguientes características:
   - Nombre del proyecto: FuncionalLab
   - Tipo de proyecto: Empty Activity
   - Lenguaje: Kotlin
   - Nombre del paquete: com.example.funcionallab
   - Mínimo SDK: API 21 o superior
2. Una vez creado el proyecto, en el panel izquierdo (Project > Android), cambiar a la vista 'Project' para ver la estructura de carpetas completa.
3. Dentro de la carpeta `app/src/main/java/com/example/funcionallab/`, hacer clic derecho y seleccionar:
   New > Kotlin Class/File > Nombre: LabFunciones.kt
   Tipo: File
4. Abrir `LabFunciones.kt` y copiar el siguiente código completo dentro del archivo:

```
package com.example.funcionallab

data class Estudiante(val nombre: String, val nota: Double)
```

```
// Extension Function 1
fun String.esNombreValido(): Boolean = this.isNotBlank() && this.length > 2

// Extension Function 2
fun Double.esAprobado(): Boolean = this >= 7.0

// Extension Property
val Estudiante.estado: String
    get() = if (nota.esAprobado()) "Aprobado" else "Reprobado"

// Higher-order function 1
fun procesarEstudiantes(estudiantes: List<Estudiante>, operacion: (Estudiante) -> Unit)
{
    for (e in estudiantes) {
        operacion(e)
    }
}

// Higher-order function 2 (inline)
inline fun filtrarEstudiantes(estudiantes: List<Estudiante>, criterio: (Estudiante) ->
Boolean): List<Estudiante> {
    return estudiantes.filter(criterio)
}

// Función principal para pruebas
fun ejecutarLaboratorio() {
    val lista = listOf(
        Estudiante("Ana", 8.5),
        Estudiante("Luis", 6.2),
        Estudiante("María", 9.0),
        Estudiante("Pedro", 5.8)
    )

    println("=== Todos los estudiantes ===")
    procesarEstudiantes(lista) {
        println("${it.nombre} - Nota: ${it.nota} - Estado: ${it.estado}")
    }

    println("\n=== Estudiantes aprobados ===")
    val aprobados = filtrarEstudiantes(lista) { it.nota.esAprobado() }
    aprobados.forEach {
        println("${it.nombre} (${it.nota})")
    }
}
```

5. Abrir `MainActivity.kt` y modificar el método `onCreate` para llamar a la función del laboratorio. Reemplazar el contenido del método `onCreate` con lo siguiente:

```
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    setContentView(R.layout.activity_main)

    // Llamada a la función principal del laboratorio
    ejecutarLaboratorio()
}
```

6. Sincronizar el proyecto: hacer clic en 'File > Sync Project with Gradle Files'.

7. Ejecutar la aplicación en un emulador o dispositivo físico. Abrir la pestaña 'Logcat' en la parte inferior de Android Studio.

8. En Logcat, filtrar por el nombre del paquete: `com.example.funcionallab`. Debería ver la salida de los estudiantes y su estado (Aprobado o Reprobado).

¿Qué ventaja tiene usar una función de orden superior como `procesarEstudiantes` en lugar de un simple bucle for? Pregunta de reflexión:

¿Qué diferencia hay entre una función de extensión y una propiedad de extensión? ¿Cuándo usarías cada una? Pregunta de reflexión:

Resultados esperados

Al ejecutar la aplicación, debería ver en Logcat una lista de estudiantes con sus notas y estados (Aprobado o Reprobado), seguida de una lista filtrada solo con los aprobados.

Desafío extra

Agregar una nueva función de extensión para el tipo `Double` llamada `esSobresaliente()` que devuelva `true` si la nota es mayor o igual a 9.0. Luego, modificar la función `filtrarEstudiantes` para que también pueda filtrar a los estudiantes sobresalientes.

Entregables

1. **Informe:** con capturas de pantallas pertinentes, con mínimo las siguientes capturas:
    - **Captura de pantalla:** Estructura de archivos en el panel Project donde se visualice el paquete de la aplicación.
    - **Captura de pantalla:** Captura de pantalla del Logcat mostrando la salida esperada.
    - **Solución:** del desafío extra, copiar el código implementado y una captura de pantalla de la ejecución de las líneas de este código.
    - **Demás puntos** que debe contener el informe como conclusiones, recomendaciones, etc.
    - **No olvide** responder cada una de las preguntas de reflexión.

Criterios de evaluación

• No olvide revisar la rúbrica de evaluación, en el EVA, para complementar su informe.